개발 리더 지원 포트폴리오 · 기준일 2026-10-05

물류 WMS 플랫폼 개발과 AI 에이전트 기반 개발 체계

C# · .NET 을 23년간 주 언어로 써 온 물류 시스템 개발자입니다. WCS · WMS 를 설계부터 설비 연동 · 배포까지 직접 구축했고, AI 코딩 에이전트를 절차와 품질 게이트 안에서 운용하는 개발 체계를 만들어 운영합니다.

지원 직무	**개발 리더**
총 경력	**23년 (2003-03 ~ 재직 중)**
연락처	**함께 제출한 이력서에 기재**

요약

저장소 9개 기준, 레거시 이관 WMS 2026-10-05 · 그 외 2026-10-01 측정

2003년부터 23년간 C# · .NET 을 주 언어로 POS · ERP · 게임 플랫폼 · 물류 시스템을 개발했고, 본부장 · 팀장으로 AGV · WMS 구축을 총괄했습니다. 현재는 물류 자동화 기업에서 WCS 아키텍처와 PLC · 표시기 설비 연동을 맡고 있습니다. 2026년 6월부터는 개인 프로젝트로 WMS 플랫폼(본체 · 연동 미들웨어 · 분석)을 Java 와 Vue 로 설계 · 구현하면서, AI 코딩 에이전트가 정해진 절차와 품질 기준 안에서 일하도록 개발 체계를 직접 설계했습니다. 3.6개월 동안 PR 2,121건 · 커밋 6,002건 · 자동 테스트 6,426건을 냈습니다.

개발 기간(저장소 기록)	PR / 머지	커밋
약 3.6개월	**2,121 / 2,082**	**6,002**
2026-06-16 ~ 10-05	측정한 5개 저장소 합	저장소 9개 합

자동 테스트	리더 역할	AI 공동작성 커밋
6,426	**3회**	**60%**
JUnit · xUnit · NUnit · pytest · JS	본부장 · 개발 팀장 · CS 개발팀장	2,403 / 4,018 (머지 제외)

PR · 커밋 · 테스트 수치는 2026-06 이후 개인 프로젝트 저장소 9개의 기록입니다.

경력 · 리더 경험

총 23년

기간	회사 · 역할	주요 업무 · 성과
2024-01 ~ 재직 중	물류 자동화 기업 · 물류자동화팀	WCS 메인 시스템 아키텍처 설계 · 구축(C# · .NET Core), PLC · 바코드 리더 · DAS/DPS 설비 연동, MQTT · REST 기반 WMS 연동, 배포 자동화. 물류센터 3곳 구축. C# 기반 시스템의 Java 전환, 자체 WMS · TMS 설계 · 개발(Spring · Oracle). 제안서 · 기능 명세서 · 인터페이스 정의서 작성, 개발 표준 · 프로세스 수립

기간	회사 · 역할	주요 업무 · 성과
2019-09 ~ 2023-12	유통 · 물류 기업 (상장사) · IT 사업 본부 본부장	AGV · WMS 설계 · 구축 총괄 PM, 중국 AGV 업체와 API 협의 · 커스터마이징. 자체 영업 솔루션(CMS) 고도화 PM(개발 2명), MS-SQL 프로시저 튜닝, 장기 · 대용량 업무를 백엔드 워커로 분산. AWS EC2 · S3 기반 구축, 상장 후 ITGC 담당
2018-01 ~ 2019-09	국제 물류 · 배송 기업 · 개발실 팀장	싱가포르 배송 · Quick 시스템 개발 · 유지보수, 풀필먼트 솔루션 신규 설계 · 구축, TMS · 배송 영역 담당. 팀원 일정 관리 · 개발 설계
2014-11 ~ 2018-01	ERP 솔루션 기업 · 연구개발팀 책임연구원	ERP 자체 솔루션 고도화(재고 수불 · 시리얼, 회계 마감 · 매출 전표). CS 개발팀장으로 일정 관리 · 선임 연구원 교육 · 긴급 요청 처리
2013-08 ~ 2014-10	게임 서비스 기업 · 웹서비스팀 과장	게임 서비스 플랫폼 · 채널링 서비스 개발, 게임 연동 API, 인증 시스템(WCF)
2012-01 ~ 2013-08	게임 개발사 · 인프라개발실 대리	그룹웨어(전자 결재 · 회의실 예약 · 커뮤니티) 구축, 게임 운영 지원 플랫폼 (봇 검출 · 권한 관리 · 통계 · 로그 연동, WPF)
2009-04 ~ 2011-06	게임 개발사 · 솔루션개발팀 대리	ASP 기반 회원 · 고객센터 시스템을 .NET 으로 리뉴얼, 휴대폰 · 이메일 인증, MOTP · 지정 PC 보안 서비스, 사내 전자 결재
2007-08 ~ 2009-04	게임 개발사 · 플랫폼실 대리	프로젝트 관리 시스템(PMS) 리뉴얼 · 유지보수, 길드 · 메신저 · 쪽지 연동 모듈 개발
2006-08 ~ 2007-08	취업 포털 기업 · 솔루션팀 대리	대학 취업 전산망 구축 · 고도화(ASP · JSP), 설정값으로 사이트를 적용하는 2.0 플랫폼 신규 구축
2005-04 ~ 2006-08	웹 에이전시 · 유지보수파트	고객사 웹사이트 유지보수, 쇼핑몰 · 웹사이트 구축(ASP · MS-SQL)

리더 경험

- 본부장(2019 ~ 2023), AGV · WMS 구축 총괄 PM 과 영업 솔루션 고도화 PM(개발 2명)을 맡았고, AGV 시스템 구축 공로로 본부장으로 승진했습니다. 상장 후에는 ITGC(IT 일반 통제)를 담당했습니다.
- 개발 팀장(2018 ~ 2019), 물류 배송 시스템 개발팀에서 팀원 일정 관리와 개발 설계를 맡았습니다.
- CS 개발팀장(2014 ~ 2018 재직 중), 일정 관리, 선임 연구원 교육, 긴급 요청 처리와 오류 추적을 맡았습니다.
- 현재(2024 ~), WCS 아키텍처 설계, 개발 표준 · 프로세스(요구사항 관리 · 형상관리 · 배포 · 테스트) 수립과 제안 · 설계 문서 작성을 맡고 있습니다.

학력 한양대학교(안산) 전자컴퓨터공학부 졸업(2003) · 정보처리산업기사 · HSK 4급 · 딥러닝 교육과정 6개월(2023-10 ~ 2024-03)

개인 프로젝트 일정 2026-06 ~ 10

- 2026-06-16 **WMS 플랫폼 본체** 첫 커밋, Spring Boot 4 + Vue 3. 09-22 까지 커밋 5,012건 · PR 1,819건 · 작업일 88일
- 2026-08-01~17 **개인 가계부**: C# / .NET + Vue 3 PWA. PR 60건
- 2026-08-23 **연동 미들웨어(EAI)** 를 본체에서 별도 서비스로 분리(설계 결정 기록 ADR). SAP ECC · 택배사 · 외부 ERP 연동. 커밋 33건
- 2026-08-29 **교정 플러그인** 시작, 에이전트의 확인되지 않은 진술과 규칙 위반을 이슈로 축적, 09-26 까지 이슈 195건 · 커밋 50건
- 2026-09-02 **레거시 이관 WMS** 시작, 기존 로직을 Spring Boot 3 + JSP + Oracle 로. 10-05 까지 커밋 699건 · PR 220건. 같은 주에 **ABC 분석 · 수요예측**(Python) PR 9건 · 커밋 12건
- 2026-09-28 **물류센터 디지털 트윈** 시작: Unity 6 관제 화면. 10-01 까지 커밋 82건, 진행 중
- 2026-09-30 **특허 검색 사이트**: FastAPI + OpenSearch + Vue 3, 특허 138,348건 색인
- 2026-10-01 특허 조사 스킬 플러그인 등록

C# · .NET 2003년부터 23년간 주 언어

경력의 대부분을 C# 과 .NET 으로 일했습니다. .NET Framework 1.0 의 WinForm POS 에서 시작해 ASP.NET · MVC · WCF · WPF(.NET Framework 3.5 · 4.6)를 거쳤고, 지금은 .NET Core 기반 WCS 와 .NET 10 · Unity 6 프로젝트를 C# 으로 만들고 있습니다.

사용 기간	C# 소스(아래 3개)	자동 테스트	.NET 세대
23년	**113,262**	**587**	**1.0 → 10**
2003 ~ 현재	줄 · 627 파일	xUnit 338 · NUnit 249	Framework 1.0 · 3.5 · 4.6 · Core 3.1 · .NET 10

기간	C# · .NET 기술	만든 것
2003 ~ 2005	.NET Framework 1.0 · WinForm · Win CE	POS(영수증 출력 · RFID), 매출 관리 시스템
2007 ~ 2011	ASP.NET · .NET Framework 3.5 · MS-SQL	게임 포털 플랫폼, ASP 기반 회원 · 고객센터 시스템의 .NET 리뉴얼, 인증 · 보안 서비스
2012 ~ 2014	ASP.NET MVC 3 · 4 · WCF · WPF · WinForm	그룹웨어, 게임 운영 지원 도구(봇 검출 · 권한 관리), 인증 시스템
2014 ~ 2019	ASP.NET MVC 4 · .NET Framework 4.6 · MS-SQL	ERP 재고 · 회계 모듈, 국제 물류 배송 · 풀필먼트 시스템

기간	C# · .NET 기술	만든 것
2019 ~ 2023	ASP.NET MVC · WinForm · .NET Framework 4.6 · ClickOnce · AWS EC2	AGV · WMS, 운송장 출력 현장 프로그램과 자동 배포, 백엔드 워커 분산 처리
2024 ~ 현재	ASP.NET Core(.NET Core 3.1) · WinForms	WCS 백엔드: REST API, PLC · 바코드 리더 · 표시기 연동
2026	ASP.NET Core(.NET 10) · EF Core 9 · Unity 6	개인 가계부, 물류센터 디지털 트윈

WCS 백엔드(재직 중 업무): C# / ASP.NET Core (.NET Core 3.1) · MariaDB · Oracle · Redis · MQTT

- 솔루션 4개 · 프로젝트 11개로 계층을 나눔, 공용 모델 → 표시기 → DAO · 서비스 → 설비 → 업무 서비스 → REST API, 설비 통신 프로그램(WinForms)
- REST API 컨트롤러 66개 · 엔드포인트 250개(기준정보 · 출고주문 · 피킹 작업 · 리포트 · 대시보드). 화면 번호마다 Controller · Adapter · Service · DTO 를 같은 구조로 두고, 외부 연동 요청은 액션 필터로 헤더 토큰을 검사
- 표시기 게이트웨이와 미들웨어 사이의 MQTT 전문(초기화 · 시각 동기화 · 점등 · 소등 · 응답 확인)을 방향별 인터페이스 2개로 정의
- 컨베이어 PLC 는 MC 프로토콜(3E 프레임) TCP 로 디바이스 값을 읽고 쓰고, 고정식 바코드 리더는 TCP 소켓으로 스캔 값과 heartbeat 를 받음. 설비 종류마다 인터페이스를 두어 구현을 교체할 수 있게 함
- 제네릭 DAO 하나로 MariaDB · Oracle 양쪽의 조회 · 등록 · 수정 · 삭제를 처리, Redis 캐시, 엑셀 업로드(EPPlus), JSON → PDF 변환(iText). MariaDB 프로시저 · 함수 · 테이블 스크립트 144개를 함께 관리

`2024-01 ~ 재직 중 · C# 395파일 80,719줄 · SQL 144파일`

개인 가계부: C# / ASP.NET Core (.NET 10) + Vue 3 PWA

- 카드 승인 문자를 붙여넣으면 금액 · 일시 · 카드사 · 할부 · 취소를 정규식으로 추출. 문장 템플릿이 아니라 항목 단위로 뽑아 카드사 17종 외 형식도 대체로 처리
- 영수증 사진은 서버의 Tesseract(kor+eng)로 OCR, 사진이 서버 밖으로 나가지 않도록 클라우드 OCR 을 쓰지 않음
- EF Core 9 · 마이그레이션 13개, 컨트롤러 12개, 사용자별 데이터 격리, PBKDF2 비밀번호, 로그인 속도 제한
- xUnit 338건(API 통합 테스트는 인메모리 DB 로 CI 에서 DB 없이 실행), Jenkins → systemd, Windows 서비스 배포 스크립트도 유지

`2026-08-01 ~ 08-17 · C# 22,201줄 · PR 60 · 커밋 99`

물류센터 디지털 트윈: Unity 6 · URP · C# (진행 중)

- 레거시 이관 WMS 의 데이터를 받아 물류센터 셀 · 표시기 상태를 3D 로 보여 주는 관제 화면. Windows 스탠드얼론 대상
- 좌표 계산 · 표시기 매핑 · MQTT 전문 해석 · 스냅샷 병합은 Unity 엔진을 참조하지 않는 Core 어셈블리에 두어, 엔진 없이 NUnit 으로 검증(테스트 249건)
- 표시기 상태는 WebSocket 위 MQTT 로 수신. 서버 주소와 MQTT 계정은 코드 · 에셋에 넣지 않고 실행 파일 옆 설정 파일과 로그인 뒤 API 로 받음
- 설계 1건 → 구현 계획 6건을 승인받은 뒤 개발. 승인된 계획서가 없는 브랜치의 코드 커밋은 훅이 차단. 브라우저 확인용 목 서버(Node) 포함

`2026-09-28 ~ 10-01 · C# 119파일 10,342줄 · 커밋 82 (AI 공동작성 79)`

C# 코드 ① DataTable → 형식 있는 목록 (WCS 백엔드)

CollectionHelper.cs (발췌) C#

```
// DataTable 을 형식 있는 목록으로 바꾸는 확장 메서드
public static List<T> vMapperList<T>(this DataTable dt) where T : new()
{
    var dataList = new List<T>();
    const BindingFlags flags = BindingFlags.Public | BindingFlags.Instance;

    // 클래스의 속성 이름과 형식(Nullable 은 안쪽 형식)
    var objFieldNames = typeof(T).GetProperties(flags).Select(item => new
    {
        Name = item.Name,
        Type = Nullable.GetUnderlyingType(item.PropertyType) ?? item.PropertyType
    }).ToList();
    …
    foreach (DataRow dataRow in dt.AsEnumerable().ToList())
    {
        var classObj = new T();
        foreach (var dtField in dtlFieldNames)
        {
            PropertyInfo propertyInfos = classObj.GetType().GetProperty(dtField.Name);
            var field = objFieldNames.Find(x => x.Name == dtField.Name);
            if (field != null)
            {
                if (propertyInfos.PropertyType == typeof(DateTime))
                    propertyInfos.SetValue(classObj, convertToDateTime(dataRow[dtField.Name]), null);
                else if (propertyInfos.PropertyType == typeof(int))
                    propertyInfos.SetValue(classObj, ConvertToInt(dataRow[dtField.Name]), null);
                …
            }
        }
        dataList.Add(classObj);
    }
    return dataList;
}
```

용도. MariaDB 와 Oracle 어느 쪽에서 읽어도 결과를 같은 모델 클래스로 받도록, 열 이름과 속성 이름을 리플렉션으로 맞춰 채웁니다. 제네릭 DAO 가 이 메서드 하나로 모든 조회 결과를 변환합니다.

C# 코드 ② 설비 인터페이스와 PLC 읽기 · 쓰기 (WCS 백엔드)

IConveyorSystem.cs · 컨베이어 구현 (발췌) C#

```
// 컨베이어 인터페이스: 설비 제조사가 바뀌면 구현만 교체
public interface IConveyorSystem
{
    void Write<T>(PLCData<T> pLCData, T value);
    T Read<T>(PLCData<T> pLCData);
    int Open();
    int Disconnected();
}

// 구현: MC 프로토콜 3E 프레임 TCP
PLCData.PLC = new Mitsubishi.McProtocolTcp(this.IP, this.Port, Mitsubishi.McFrame.MC3E);

// 디바이스 객체는 호출하는 쪽에서 한 번 만들어 재사용(매번 new 하지 않음)
public void Write<T>(PLCData<T> pLCData, T value)
{
    var e = pLCData;
    e[0] = value;
    e.WriteData();
}

public T Read<T>(PLCData<T> pLCData)
{
    pLCData.ReadData();
    return pLCData[0];
}
```

이유. 업무 코드는 인터페이스만 알고, 제조사별 통신(MC 프로토콜 등)은 구현 클래스에 둡니다. 디바이스 주소 객체를 호출 쪽에서 한 번 만들어 넘기게 해, 반복 호출에서 객체가 계속 생기지 않게 했습니다.

C# 코드 ③ MQTT 패킷 인코딩 (디지털 트윈)

MqttPacketCodec.cs (발췌) C#

```
// MQTT 3.1.1 중 구독에 필요한 패킷만 인코딩한다. 발행(PUBLISH)은 만들지 않는다.
public static class MqttPacketCodec
{
    public static byte[] EncodeSubscribe(int packetId, IEnumerable<string> topics, int qos)
    {
        var header = new List<byte> { (byte)(packetId >> 8), (byte)packetId };
        var payload = new List<byte>();
        foreach (var t in topics)
        {
            WriteString(payload, t);
            payload.Add((byte)qos);
        }
        return Build(0x82, header, payload);
    }

    public static byte[] EncodePingReq() => new byte[] { 0xC0, 0x00 };
    …
    static byte[] Build(byte first, List<byte> header, List<byte> payload)
    {
        var packet = new List<byte> { first };
        int n = header.Count + payload.Count;
        do                                  // 남은 길이: 7비트씩 가변 길이
        {
            int d = n % 128;
            n /= 128;
            packet.Add((byte)(n > 0 ? d | 0x80 : d));
        } while (n > 0);
        packet.AddRange(header);
        packet.AddRange(payload);
        return packet.ToArray();
    }
}
```

이유. 관제 화면은 표시기 상태를 받기만 하므로 구독에 필요한 패킷만 직접 구현했습니다. Unity 엔진을 참조하지 않는 어셈블리에 두어 엔진 없이 NUnit 으로 바이트 단위까지 검증합니다.

대표 프로젝트: WMS 플랫폼 본체 · 연동 미들웨어 · 분석, 3개 서비스

물류센터의 입고 · 재고 · 출고 · 반품 · 구매 · 설비 운영을 한 화면 체계에서 다루는 WMS 입니다. 본체 하나에 업무를 모두 넣고, 외부 시스템 연동과 통계 · 예측은 각각 별도 서비스로 분리했습니다.

본체 Spring Boot 4.1 · Java 25 · Vue 3.5 · TypeScript · MyBatis 4.1

Java 본문	Vue · TS	자동 테스트	PR / QA 리포트
149,810	**203,470**	**4,361**	**1,819 / 126**
줄 · 1,773 파일	줄 · 259 + 240 파일	JUnit 3,976 · Vitest 385	1,789 머지

업무 범위(패키지 기준)

- **출고**: 웨이브, 할당 전략 엔진, 피킹, DPS · DPC · DAS 설비, 박스 · 검수 · 출하 · 배차, 크로스도킹, 매장주문
- **재고**: 실사, 보충, 이동, 창고간 이동, 조정, 로트, 파렛트 원장, RFID 단위
- **입고 · 구매 · 반품 · 운영**: 입고 재배치, 구매, 반품 · 품질 통지, 도크 운영, 배차
- **기준정보 34개 하위 모듈**: 위치 · 존 · 설비 · 표시기 · 카트 레이아웃 · 적치 · 할당 전략 · 운송사 · 차량 · RFID 리더
- **그 외**: PDA 화면 18개, 도크 예약 포털, 사내 채팅, 게시판, 알림, 출력 라우팅, 감사 로그, 다국어 4개(ko · en · ja · zh)

설계 포인트

- **다중 DB**: MariaDB · Oracle · PostgreSQL 을 환경 변수로 고르고, 테이블 168개의 DDL 을 MariaDB · Oracle 양쪽으로 유지. 벤더별 SQL 은 MyBatis `databaseId` 로 분기
- **인증**: 불투명 Bearer 토큰(JWT 아님). 저장소를 메모리 · DB · Redis 중에서 설정으로 교체. 사용자별 메뉴 권한, 대리 결재
- **실시간**: STOMP WebSocket(사내 채팅), MQTT(표시기 점등 · 가상 표시기)
- **스케줄**: 크론을 DB 에 등록해 실행 중 변경, ShedLock 으로 다중 인스턴스 중복 실행 방지
- **데이터 접근**: MyBatis 만 사용. JPA · QueryDSL 은 백로그 결정으로 제거(`@Entity` 0)
- **배포 단위**: Vue 빌드를 jar 안에 넣어 서비스 하나로 배포

설계 결정 기록(ADR) 11건, 상세설계 14건, 설계안 문서 다수 등 문서 111개를 저장소에 두고 결정과 근거를 날짜와 함께 남겼습니다.

연동 미들웨어(EAI) Spring Boot 4.1 · Java 25 · MariaDB · Vue 3

연동 대상

- **SAP ECC**: 인터페이스 테이블 + MariaDB 프로시저 4개, 인터페이스 ID 10종의 JSON 스키마
- **택배사 API**: 주소 정제 · 송장 발주, 크론 작업 2개
- **기준정보 수신 13종**: 상품 · 거래처 · 창고 · 위치 · 브랜드 등. 매핑 → 변환 → 참조 해소 → 검증 → 자연키 upsert 5단계, 부분 성공 허용
- **전표 수신 2종**: 입하예정 · 출고오더

`Java 11,321줄 · 테스트 110건 · PR 13건`

신뢰성 설계

- 수신은 10초 간격 폴링, 건별 try/catch 로 한 건의 실패가 다음 건을 막지 않음
- 처리기는 멱등(재고 = 스냅샷 차이, 자재 = upsert)으로 두어 at-least-once 전달
- 재시도 횟수 기록, 수동 재처리 상한(기본 5회)을 SQL 조건으로 강제
- 아웃박스 테이블로 이벤트 발행, 오류 알림은 중복 키 · 쿨다운으로 묶음
- WMS 도메인 테이블을 직접 쓰지 않고 본체 REST 로만 변경
- 통합 테스트가 하나라도 건너뛰면 빌드 실패, 운영 반영 전 스키마 게이트

ABC 분석 · 수요예측 Python 3.12 · pandas · scikit-learn · FastAPI · systemd timer

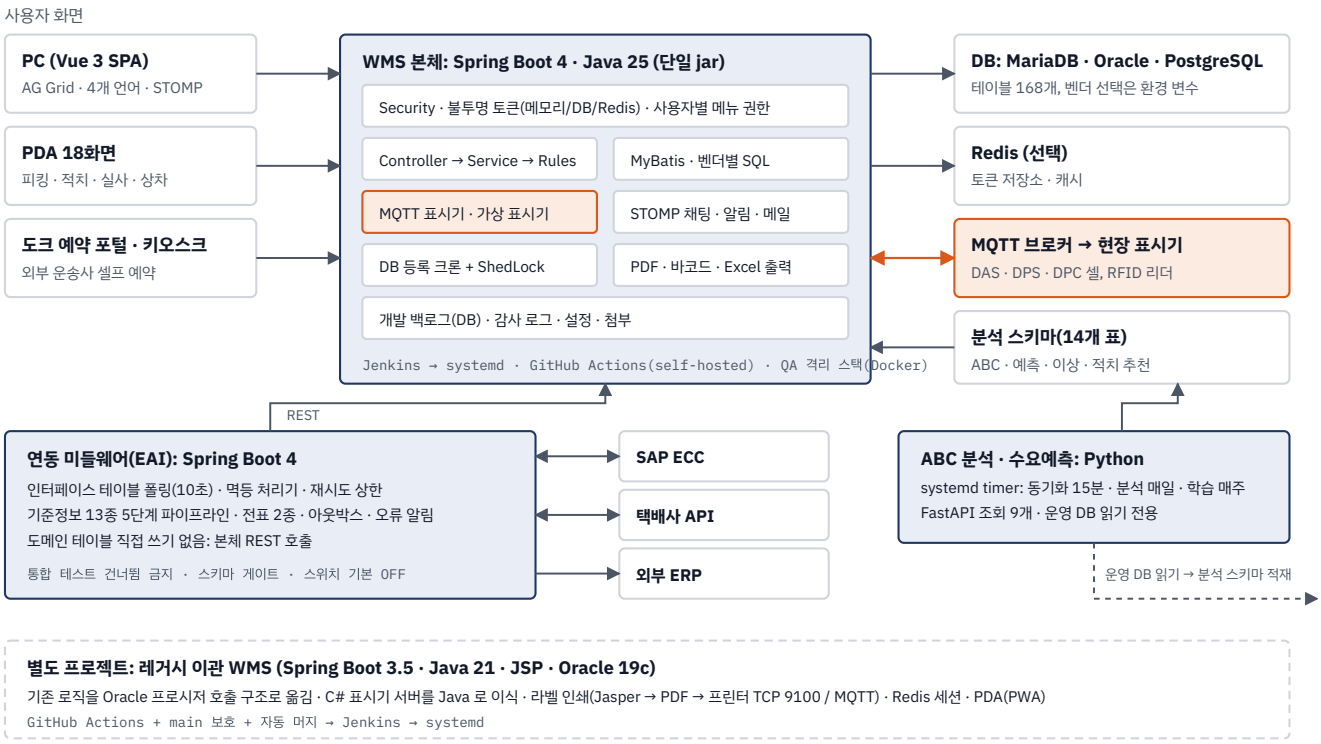
계산 내용

- ABC 파레토(수량 · 금액 · 터치 수, 80/15 기준), XYZ 수요 안정성, Syntetos-Boylan 수요 패턴 분류
- 회전율 · 재고일수 · 평가액 · 충족률
- SKU 일별 수요 28일 예측, 그래디언트 부스팅, 비교 기준으로 직전 28일 합과 Croston SBA
- 미래 ABC 등급과 전이, 이상 탐지(MAD z-score + IsolationForest), 적치 위치 추천 상위 5개

운영 방식

- 운영 DB 의 등록 마스터를 읽어 별도 스키마(14개 표)에 적재, 본체 화면에서 조회
- 동기화 15분마다, 분석 매일, 학습 매주, systemd timer
- 운영 거래가 52건뿐이어서 실제 마스터 위에 2년치 거래를 합성해 모델을 검증, 실데이터 전환 옵션 유지
- Python 3,173줄 · pytest 53건 · ruff · pyright

아키텍처 플랫폼 전체 구성



레거시 이관 WMS Spring Boot 3.5 · Java 21 · JSP · Oracle 19c · 2026-09 ~ 10

운영 중인 기존 WMS 의 업무 로직을 Oracle 프로시저 호출 구조와 JSP 화면 체계로 옮기는 프로젝트입니다. C# · MariaDB · iBATIS · Vue · 상용 리포트로 된 원본을 Java · Oracle 구조로 옮기고, 원본 대비 변경점을 문서로 남겼습니다.

Java 본문	JSP · 자체 JS	자동 테스트	PR / 이슈
56,207	**125 · 232**	**1,315**	**220 / 241**
줄 · 707 파일	파일 (JS 37,967줄)	JUnit 1,020 · JS 295	이슈 235건 해결

이 저장소에서 새로 설계 · 구현

- 위치재고 원장(입 · 출고 이력 테이블과 증감 규칙)
- 입고적치 · 재고조사 · DPS 보충 · 총량피킹 업무 흐름
- 요율 관리(운임 기본 요율 · 납품처 등급 가산 · 포장자재 청구 단가, 적용 시작일 이력)와 박스당 운임 조회
- 박스 품목별 적입 수량(여섯 방향 회전 배치 · 부피 · 중량 기준)을 DPC 적재판정 · DAS 점등 수량에 반영
- 3D 현황(three.js): 창고 · 위치재고 · 재고현황 · 셀 재고 · 로케이션배치도
- 로그인 세션 Redis 저장(재배포 후에도 로그인 유지)
- 라벨 인쇄 경로(프린터 TCP 9100 직결 / PC 클라이언트 MQTT), 프린터 그룹 병렬 출력 · 비트맵(ZPL) 출력
- 모바일 메인화면 KPI · DB 기반 메뉴 · 권한 · 동적 스케줄러
- CI/CD 파이프라인과 품질 게이트, 에이전트 작업 체계

기존 시스템에서 이관 · 재구성

- C# 표시기 MQTT 서버 → Java 표시기 모듈
- MariaDB 프로시저 · 함수 129개 → Oracle 19c
- Spring 레거시(iBATIS) 공통 구조 → Spring Boot 3 + MyBatis 3
- 본체(Boot 4 + Vue 3)의 표시기 모듈 · DPC 카트 화면 → Boot 3 + JSP
- 상용 리포트(.ezg) · .NET 라벨 → 라벨 디자이너 / JasperReports
- Redis 캐시 모듈 이식(캐시 9종, TTL 30분)

1 **두 가지 데이터 접근 방식을 함께 둠.** 이관한 화면은 기존과 같은 시그니처의 DataMap · CommonDAO 로 옮겨 동작 차이를 줄이고, 새 기능은 타입이 있는 Mapper · DTO 로 작성했습니다.

2 **DB 밖 작업은 커밋 뒤에만.** 캐시 무효화 · 표시기 발행 · 외부 WMS 전송은 트랜잭션 커밋이 끝난 뒤 실행해, 롤백 시 DB 와 바깥 상태가 어긋나지 않게 했습니다.

3 **표시기 전문은 큐 하나로 보냄.** 게이트웨이가 연속 전문을 놓치는 현장 제약(150ms 간격)을 발행 경로 하나에서만 지키게 해 순서와 재시도를 보장했습니다.

4 **프로시저 호출 코드는 생성.** Oracle 프로시저 시그니처를 읽어 DTO · Mapper · XML · Service 를 만드는 생성기로 129개 프로시저의 호출 계층을 같은 형태로 맞췄습니다.

5 **라벨 출력 방식은 측정한 뒤 정함.** 비트맵 출력은 장당 조립 3.5~5.5ms, 전문 38~121KB 로 측정했고 100Mbps 랜에서 100장 전송 0.3~1.0초로 환산했습니다. 프린터 그룹은 프린터마다 스레드 하나로 동시에 보내고, 실패한 라벨은 같은 그룹의 다음 프린터로 한 번 더 보냅니다.

6 **부하테스트로 병목과 누수를 찾아 고침.** 전체 화면(PC 60 · 모바일 17) 조회 부하에서 Oracle JDBC 기본 fetch size(10) 때문에 1,276행 조회에 DB 왕복이 128번 생기는 것을 찾아 500 으로 바꿨습니다. 같은 조건에서 PC 동시 100명 처리량이 초당 425건에서 1,351건으로, p95 가 682ms 에서 202ms 로 바뀌었고, 3D 탭을 열고 닫을 때마다 남던 DOM 노드 1,020~1,333개는 0개가 됐습니다.

언어 · 플랫폼 전환 참조 소스를 읽고 동작을 맞춰 옮긴 10건

원본	대상	범위	상태
C# (.NET) 표시기 MQTT 서버	Java · Spring Boot 3	참조 소스 8파일 5,194줄. 매니저 명령 24개 모두 이식, 원본의 REST 왕복 조회를 DB 직접 조회로 단순화	운영 코드 반영
C# WMS 전송 프로그램	Java (RestClient)	할당 결과 · 피킹 실적 전송, 트랜잭션 커밋 뒤 전송	구현 · 기본 꺼짐
MariaDB 프로시저 · 함수	Oracle 19c	129개. 암시적 결과셋 → OUT SYS_REFCURSOR, 문법 규칙표 작성, 변환 중 원본 결함 10건 보정 기록	변환 완료
프로시저 시그니처	Java 호출 계층	생성기로 DTO 129개와 Mapper · XML · Service 생성	완료
MariaDB 스키마 (본체)	Oracle · PostgreSQL	테이블 168개 DDL 을 MariaDB · Oracle 양쪽으로 유지, 벤더별 SQL 분기, 벤더 선택은 환경 변수	완료
Spring 레거시 (iBATIS)	Spring Boot 3 + MyBatis 3	DataMap · CommonDAO · BaseController 를 같은 시그니처로 이식	완료
Spring Boot 4 + Vue 3	Boot 3 + JSP	표시기 모듈(Jackson 3 → 2, DTO → record), DPC 카트 화면	완료
상용 리포트 (.ezg 바이너리)	라벨 디자이너 시드	33개 중 31개 이관. 변환 도구가 없어 파일 바이트를 직접 읽어 양식을 복원	이관 완료
.NET 리포트 라벨	JasperReports 7 → PDF · 비트맵	JRXML 4종, PDF 미리보기. 쪽 이미지를 흑백 비트맵(ZPL ^GFA)으로 바꿔 라벨프린터로 전송	실프린터 확인 진행 중
Bootstrap 4 테마 (SCSS)	Bootstrap 5.3 + JSP	테마 토큰 · 컴포넌트, 아이콘 세트 교체	완료

기타 프로젝트

특허 검색 사이트: FastAPI + OpenSearch + Vue 3

- 수집기가 적재한 특허 138,348건(색인 약 1.15GB)을 검색. MariaDB 10.11 에 n-gram 파서가 없어 OpenSearch 한국어 형태소 분석기(nori)를 택함
- 검색은 OpenSearch 에서 출원번호를 얻고 상세는 MariaDB 에서 읽는 2단계, 전체 · 증분 색인기
- 공개 API 는 키의 SHA-256 해시만 저장, 키별 분당 60회 제한
- 약 1,100줄, Jenkins → systemd. 코드는 DB 에 쓰지 않는 읽기 전용

`2026-09-06 ~ 09-30 · Python · Vue · 커밋 13 (AI 공동작성 13)`

기술 역량

C# · .NET (주 언어)	23년. ASP.NET Core(.NET Core 3.1 · .NET 10) · ASP.NET MVC · WCF · WPF · WinForms · EF Core · Unity 6. 최근 3개 프로젝트 C# 약 11만 줄
백엔드 Java	Spring Boot 4.1 · Java 25(본체 · EAI), Spring Boot 3.5 · Java 21(이관). Java 본문 약 22만 줄
백엔드 Python	FastAPI · SQLAlchemy · pandas · scikit-learn
프론트엔드	Vue 3 · TypeScript · Vite · Pinia · AG Grid · Chart.js · vue-i18n 4개 언어(약 20만 줄), JSP · jQuery · SlickGrid · three.js
데이터베이스	MariaDB · Oracle 19c · PostgreSQL 다중 DB, 프로시저 변환 129개, MyBatis XML 295개, DDL 168 테이블 양 벤더 유지
검색 · 분석	OpenSearch 한국어 형태소, ABC/XYZ · 수요예측(그래디언트 부스팅 · Croston) · 이상 탐지
현장 설비 · 연동	MQTT 표시기(DAS · DPS · DPC), 컨베이어 PLC(MC 프로토콜), 고정식 바코드 리더(TCP), RFID 리더, 라벨프린터 TCP 9100(ZPL 텍스트 · 비트맵), SAP ECC · 택배사 · 외부 ERP 인터페이스
모바일 · 현장 단말	PDA 화면 18 + 35개, PWA(가계부 · PDA), 키오스크, 도크 예약 포털
CI/CD · 운영	GitHub Actions(self-hosted 러너, 워크플로 7개) · Jenkins · systemd 배포 · 헬스체크, Testcontainers · Docker QA 스택
품질	자동 테스트 6,426건, QA 리포트 126 + PR 본문 QA 절 95, 테스트 건너뜀 금지 · 무작위 순서 · 경고=오류
AI 협업 체계	작업 절차 · 훅 · 게이트 설계, 자체 MCP 서버 6개, 교정 플러그인, 커밋 4,018건 중 2,403건 AI 공동작성
문서화	저장소 문서 147개 이상(ADR 11 · 상세설계 14 · 설계안 · 운영 절차), 결정과 근거를 날짜와 함께 기록

보안	불투명 토큰 · BCrypt · PBKDF2, CSRF, 메뉴 권한 인터셉터, API 키 해시 · 속도 제한, 보안 헤더

핵심 코드 레거시 이관 WMS 에서 발췌, … 는 생략 표시

① C# 원본 → Java 이식: 표시기 발행

XMQTTController.cs (원본) C#

```
internal void Publish(string topic, string msg,
                      Action<string, ushort> callBack
= null)
{
    try
    {
        if (MQClient == null) return;
        if (MQClient.IsConnected)
        {
            var result = MQClient.Publish(topic,
                Encoding.UTF8.GetBytes(msg));
            System.Threading.Thread.Sleep(150);
            //동시에 보내면 정보가 소실된다.
            if (callBack != null)
                callBack(msg, result);
        }
    }
    catch (Exception ex)
    {
        log.Error($"Publish {ex.Message}");
    }
}
```

OutboundPublishQueue.java (이식) Java

```
// 호출측은 큐에 넣고 즉시 돌아감
public boolean enqueue(String topic, String payload,
…) {
    …
    boolean accepted = queue.offer(
        new PublishRequest(topic, payload,
Instant.now(), listener));
    …
}

// 소비 스레드 하나: 순서(점등 → 소등) 유지
private void drainLoop() {
    while (running.get() || !queue.isEmpty()) {
        request = queue.poll(200,
TimeUnit.MILLISECONDS);
        …
        deliver(request);
        if (!pause(intervalMillis)) return; // 기본
150ms
    }
}

// 실패하면 같은 자리에서 재시도(순서 유지)
private void deliver(PublishRequest request) {
    for (int attempt = 0; attempt <= maxRetries;
attempt++) {
        if (mqttClient.publish(request.topic(),
request.payload())) {
            … return;
        }
        …
    }
    droppedCount.incrementAndGet();
    degraded.set(true); // 화면에 "보내지 못함" 안내
    …
}
```

바꾼 점. 원본은 호출한 스레드가 150ms 를 직접 쉬고, 실패하면 로그만 남깁니다. 이식하면서 간격 값(현장 실측 150ms)은 그대로 두고, 발행 경로를 큐 하나로 모아 소비 스레드가 순서대로 보내게 했습니다. 브로커가 잠시 끊기면 같은 전문을 다시 시도하고, 끝내 실패하면 화면이 알 수 있도록 상태를 남깁니다. 간격 · 큐 용량 · 재시도 횟수는 설정으로 뺐습니다.

② 커밋 뒤에만 DB 밖 작업 실행

AfterCommit.java Java

```java
/** 트랜잭션 커밋 후에만 실행, 캐시 무효화·표시기 MQTT 발행·외부 WMS 전송 */
public final class AfterCommit {

    public static void run(Runnable action) {
        if (TransactionSynchronizationManager.isSynchronizationActive()) {
            TransactionSynchronizationManager.registerSynchronization(
                new TransactionSynchronization() {
                    @Override
                    public void afterCommit() {
                        action.run();
                    }
                });
        } else {
            action.run();    // 트랜잭션 밖(테스트·배치)은 즉시 실행
        }
    }
}
```

이유. 커밋 전에 캐시를 지우거나 표시기를 켜면, 그 뒤 롤백될 때 DB 는 옛 상태인데 바깥만 새 상태로 남습니다. 되돌릴 주체가 없으므로 실행 시점을 커밋 뒤로 고정했습니다.

③ DPC 카트 적재판정 · 박스 점유율

BoxOccupancy.java · DpcFitJudge.java — Java 21 record

```
public enum Reason { OK, NO_DIMENSION, DIMENSION, UNITS_PER_BOX, VOLUME, WEIGHT }

/** 품목 하나의 점유율 항: 지정 품목은 수량 ÷ MAX_QTY, 미지정 품목은 수량 ÷ 용량 */
public static BigDecimal term(Box box, Integer maxQty, Dim dim, long qty) {
    …
    if (maxQty != null && maxQty > 0) {
        return BigDecimal.valueOf(qty).divide(BigDecimal.valueOf(maxQty), SCALE, RoundingMode.DOWN);
    }
    long capacity = capacity(box, dim);   // min(여섯 방향 배치 수량, ⌊유효부피 ÷ 품목 부피⌋)
    …
    return BigDecimal.valueOf(qty).divide(BigDecimal.valueOf(capacity), SCALE, RoundingMode.DOWN);
}

/** 품목을 여섯 방향(축 순열)으로 돌려 각 축에 놓이는 몫의 곱 중 최대 */
public static long placeQty(Box box, Dim dim) {
    …
    int[][] orders = { {0,1,2}, {0,2,1}, {1,0,2}, {1,2,0}, {2,0,1}, {2,1,0} };
    long best = 0;
    for (int[] o : orders) {
        best = Math.max(best, (b[0] / d[o[0]]) * (b[1] / d[o[1]]) * (b[2] / d[o[2]]));
    }
    return best;
}

// DpcFitJudge.judge: 치수 누락 → 낱개 크기 → 점유율 합 → 중량 순으로 처음 걸린 사유
if (occupancy.compareTo(BigDecimal.ONE) > 0) {
    return designated ? Reason.UNITS_PER_BOX : Reason.VOLUME;
}
if (weight.compareTo(BigDecimal.valueOf(box.weightLimitG())) > 0) return Reason.WEIGHT;
return Reason.OK;
```

이유. 주문 하나가 카트 칸 박스에 들어가지 않으면 설비로 보내지 않고 수작업으로 돌립니다. 박스마다 품목별 적입 수량을 지정할 수 있고, 지정이 없으면 여섯 방향 회전 배치 수량과 부피 기준 중 작은 값을 용량으로 씁니다. 같은 규칙이 피킹확정 Oracle 프로시저에도 있어, 항마다 소수 12자리에서 내림(SQL `TRUNC(x, 12)` · Java `RoundingMode.DOWN`)해 두 구현의 결과를 맞췄습니다. 판정은 가장 작은 칸 기준이라 어느 칸에 배정돼도 들어가고, DAS 표시기도 이 규칙으로 남은 개수만큼만 점등합니다.

④ MariaDB → Oracle 프로시저 호출 규약

변환 전 · 후 호출 SQL

```
-- MariaDB: 본문의 SELECT 가 그대로 결과셋
CALL SP_MST_PUB002_GET(:json, @c, @cl, @w, @r, @m);

-- Oracle: 결과셋을 OUT 커서로 받음
DECLARE
  v_c  VARCHAR2(10);  v_cl VARCHAR2(10);  v_w
VARCHAR2(20);
  v_r  VARCHAR2(4000); v_m VARCHAR2(4000);
  v_cur1 SYS_REFCURSOR;  v_curR SYS_REFCURSOR;
BEGIN
  SP_MST_PUB002_GET(:json, v_c, v_cl, v_w, v_r, v_m,
                    v_cur1, v_curR);
END;
```

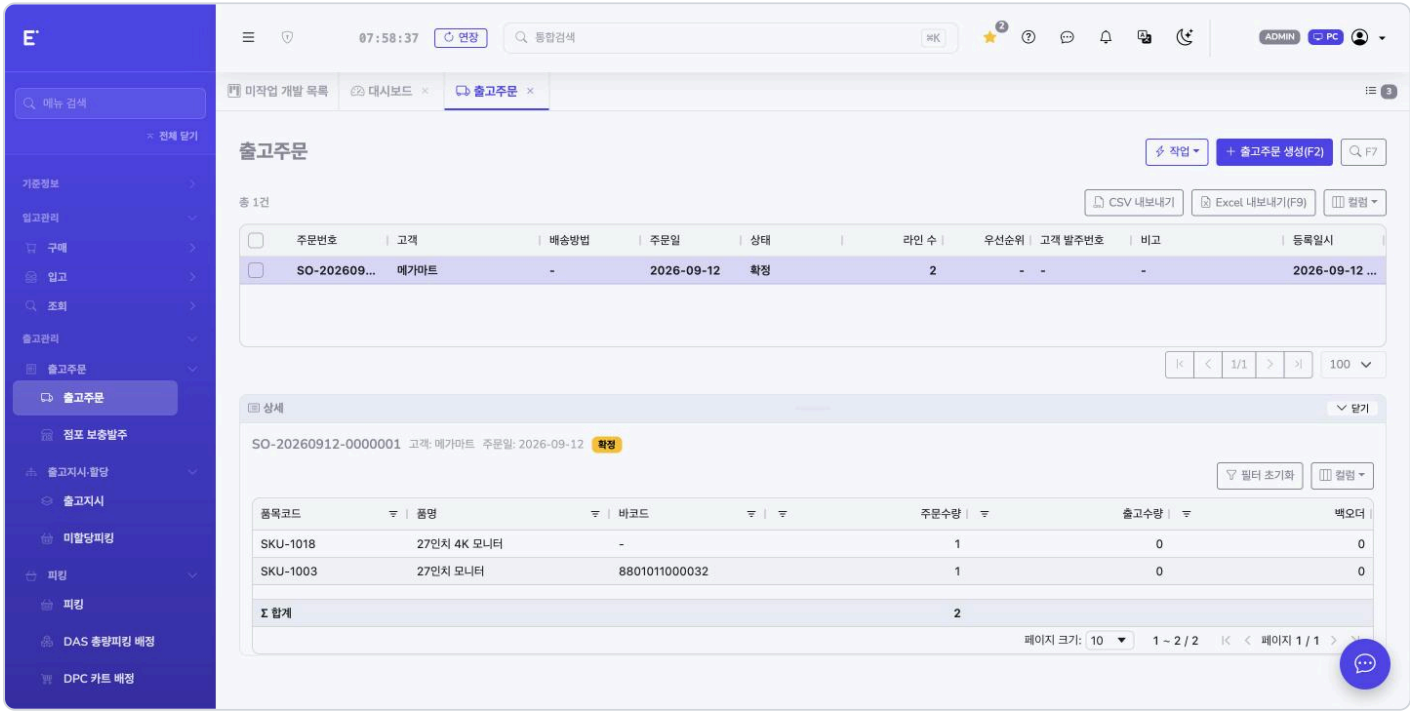
구문 변환 규칙표(발췌) SQL

```
-- MariaDB                  → Oracle 19c
LIMIT 1                     → FETCH FIRST 1 ROWS ONLY
IF(c, a, b)                 → CASE WHEN c THEN a ELSE
b END
GROUP_CONCAT(x ORDER BY y) → LISTAGG(x, ',')
                              WITHIN GROUP (ORDER BY
y)
INSERT IGNORE … SELECT      → MERGE … WHEN NOT MATCHED
CREATE TEMPORARY TABLE      → 전역 임시 테이블(GTT)
SELECT … INTO (무결과)      → MAX() 로 감싸 1행 보장
```

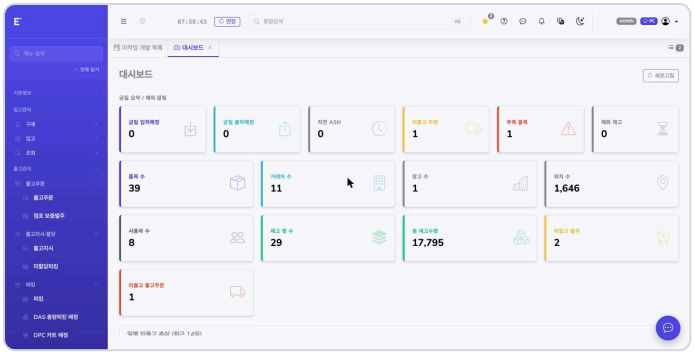
원칙. 기존 IN/OUT 파라미터의 이름과 순서는 그대로 두고, 결과셋 커서만 맨 뒤에 붙였습니다. 예외가 나도 커서를 빈 결과로 열어 클라이언트 오류(ORA-01001)를 막았습니다. 원본에서 정의되지 않은 변수 · 컬럼을 참조하던 10곳은 의도를 추정해 고치고 파일마다 표시했습니다.

주요 화면 개발 환경 화면

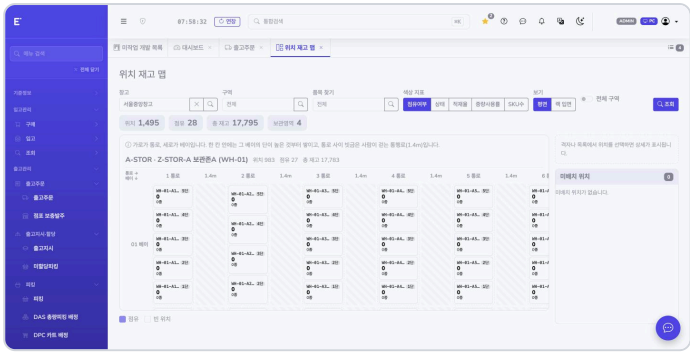
WMS 본체 (Vue 3 · TypeScript · AG Grid)



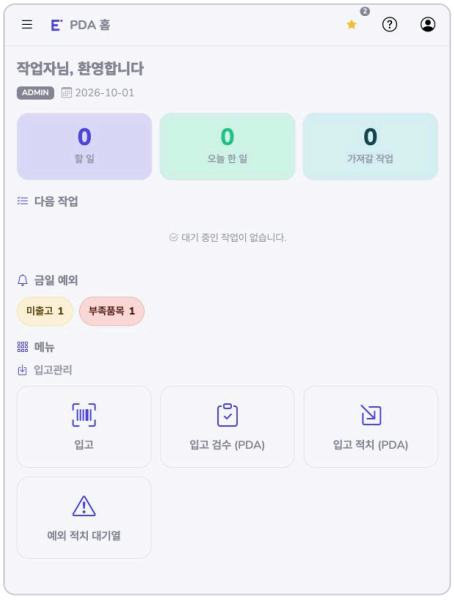
출고주문. 목록 그리드에서 주문을 고르면 아래에 품목 상세와 합계 행을 표시. 화면은 탭으로 열고 CSV · Excel 내보내기와 컬럼 설정은 모든 그리드 공통



대시보드. 금일 요약과 예외 · 알림 카드, 기준정보 · 재고 수량



위치 재고 맵. 통로 · 베이 · 단 배치대로 재고를 그리고 색상 지표(점유 · 상태 · 적재율 · 중량)와 평면 · 랙 입면 보기를 전환



PDA 홈. 할 일 · 금일 예외와 업무별 메뉴 타일

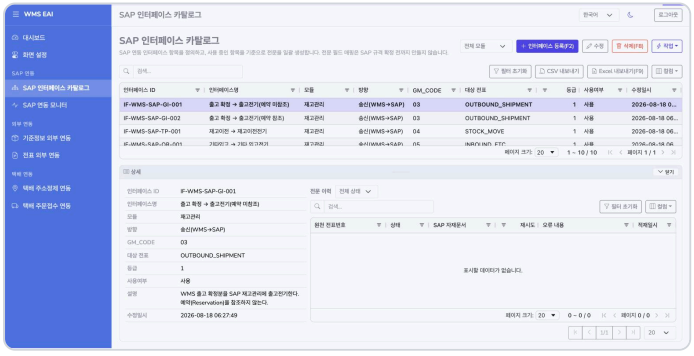


PDA 입고. 입하예정 번호와 바코드 스캔, 유통기한은 +30 · +90 · +180 · +365 버튼으로 입력



PDA 피킹. 작업지시번호 스캔으로 시작하고 완료 건수 · 수량을 표시

연동 미들웨어(EAI) (Vue 3 · AG Grid)

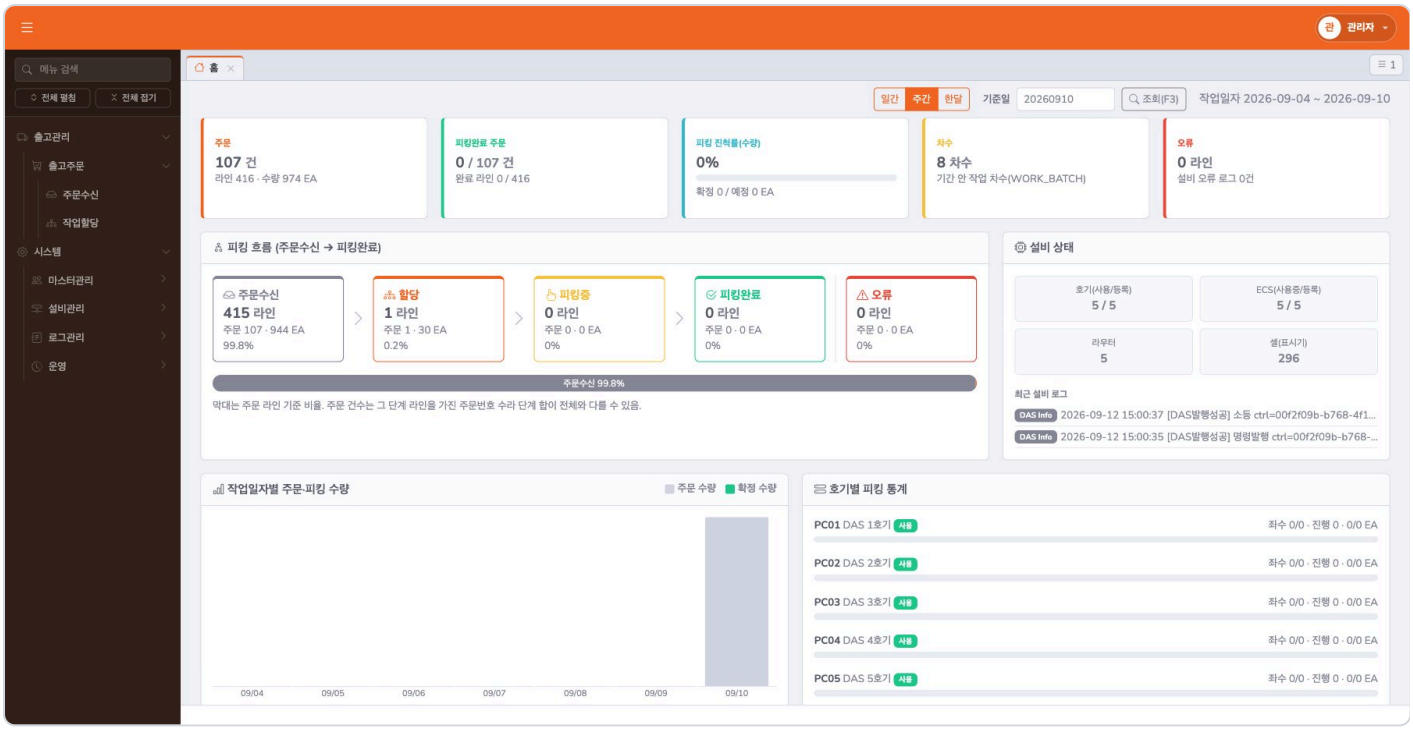


SAP 인터페이스 카탈로그. 인터페이스 항목을 정의하고 항목별 전문 이력(상태 · 재시도 · 오류 내용)을 조회

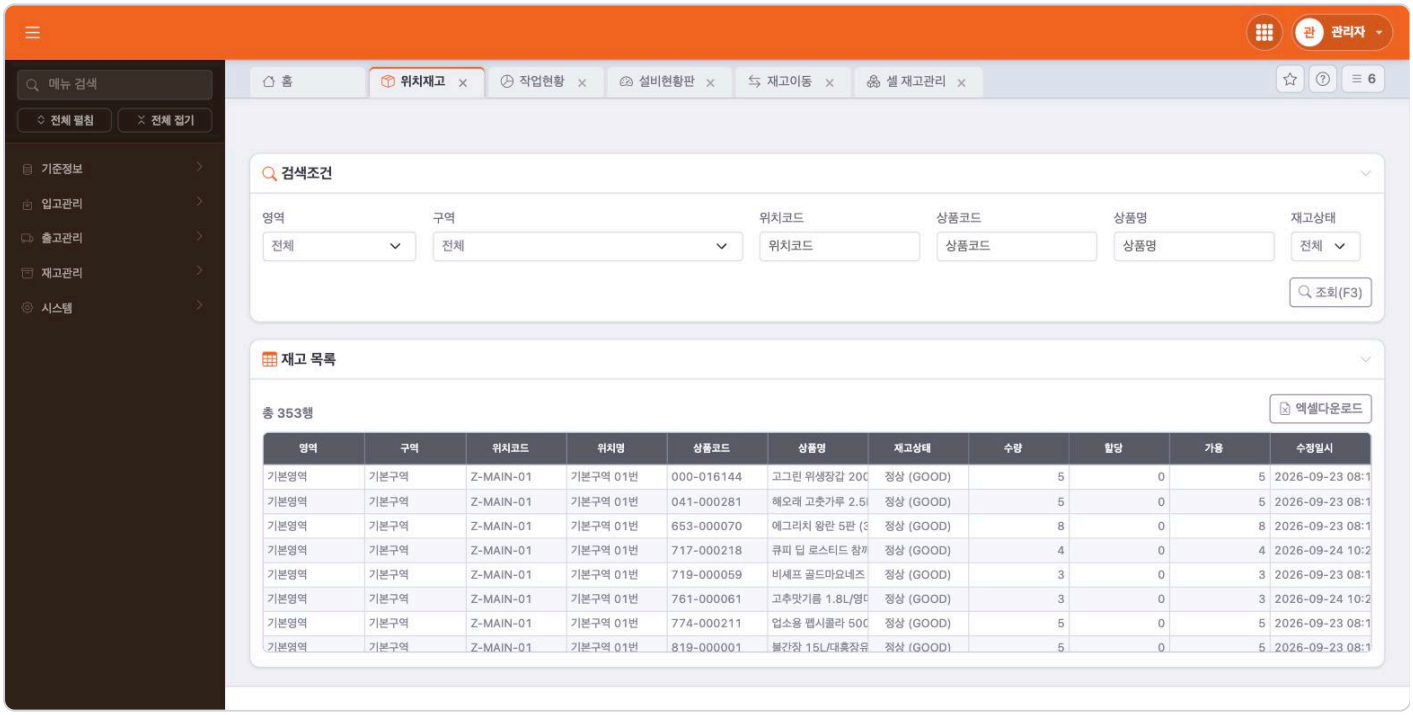


기준정보 외부 연동. 기준정보 13종마다 같은 행 판정 기준과 필드 매핑을 화면에서 정의하고 수신 이력을 조회

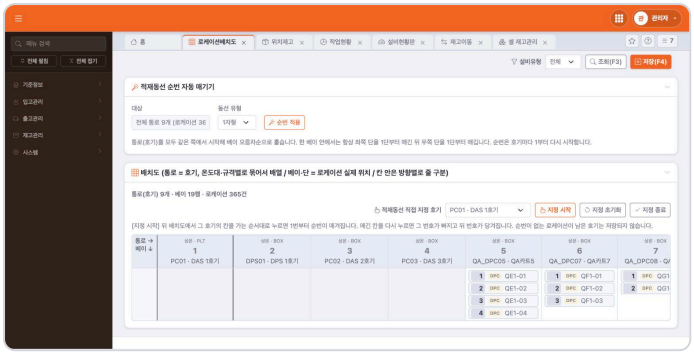
레거시 이관 WMS (JSP · SlickGrid)



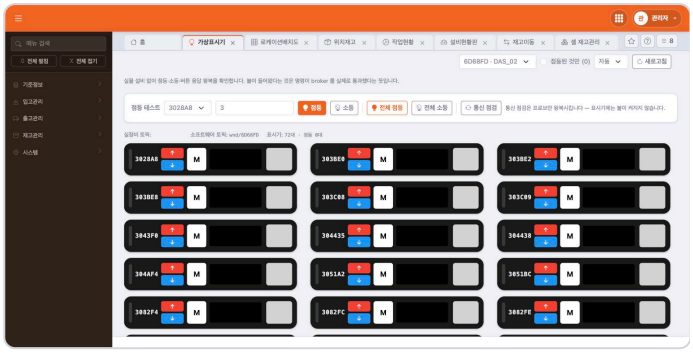
레거시 이관 WMS 홈 대시보드, 주문 · 피킹 KPI, 단계별 흐름, 설비 상태, 호기별 피킹 통계



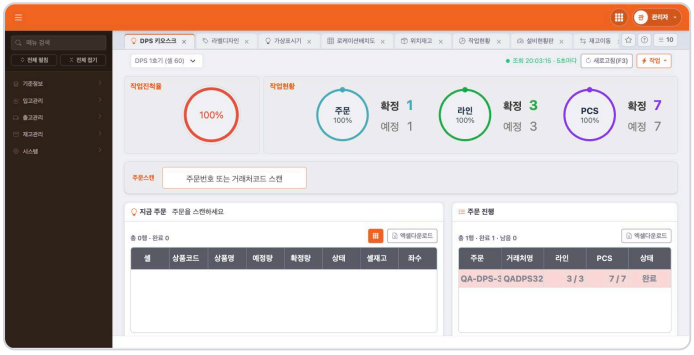
PC 그리드 화면(위치재고). 오픈소스 그리드 SlickGrid 위에 조회 · 저장 · 엑셀 · 열 설정 공용 모듈을 얹어 화면마다 같은 동작



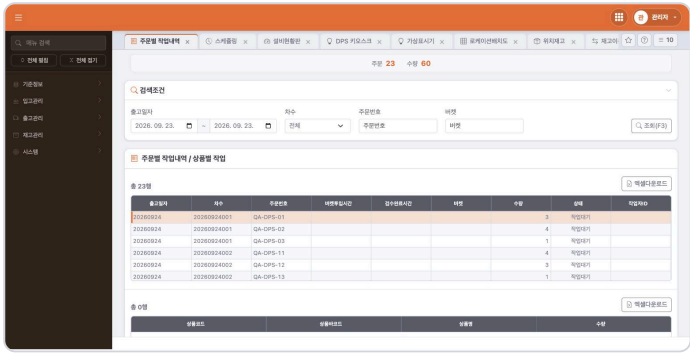
로케이션배치도. 통로(호기) · 베이 · 단을 실제 위치대로 그리고 적재동선 순번을 눌러서 지정



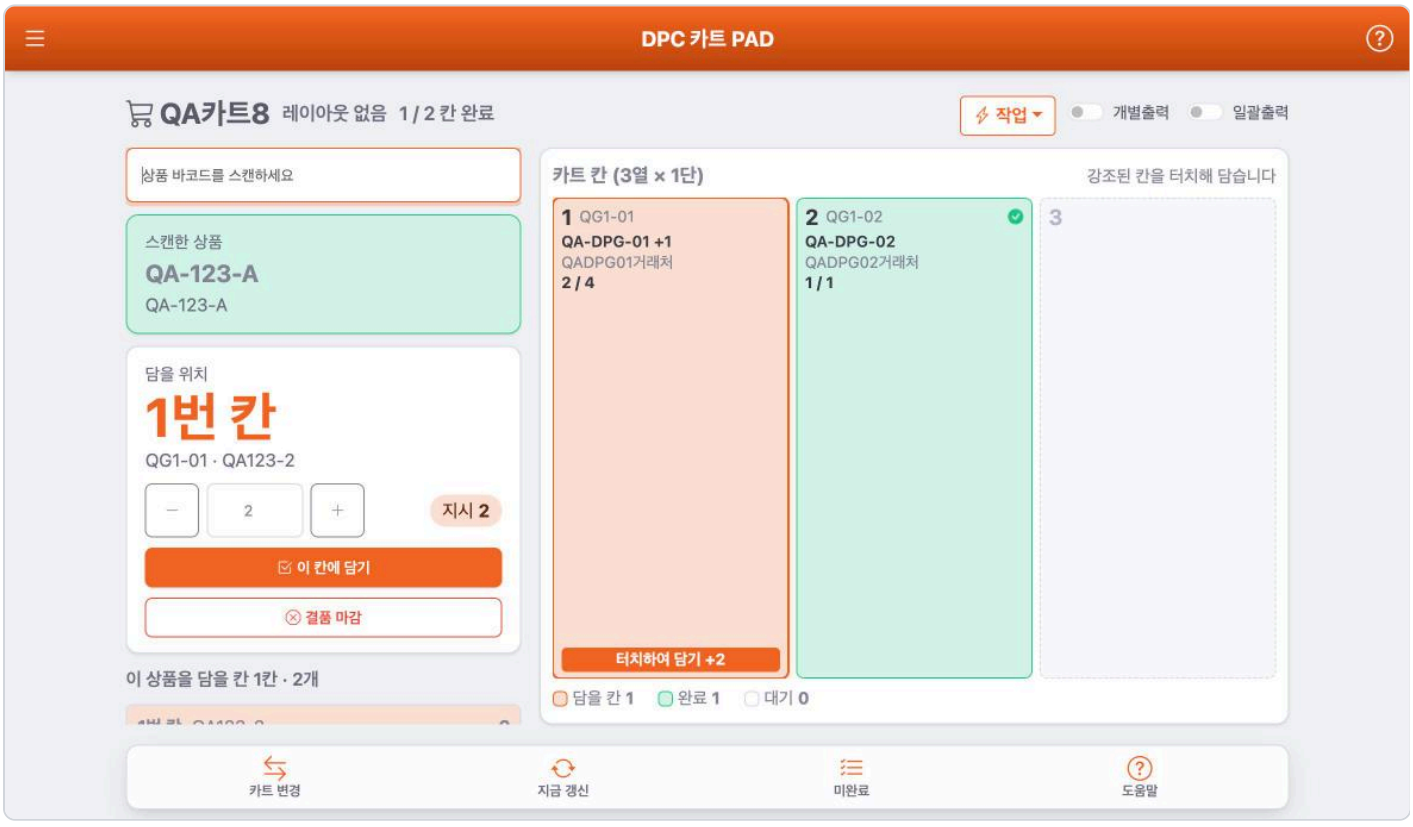
가상표시기. 실물 설비 없이 브라우저 MQTT 로 점등 · 소등 · 버튼 응답 왕복을 확인



DPS 키오스크. 주문을 스캔하면 해당 셀 표시기가 켜지고 진행률을 5초마다 갱신



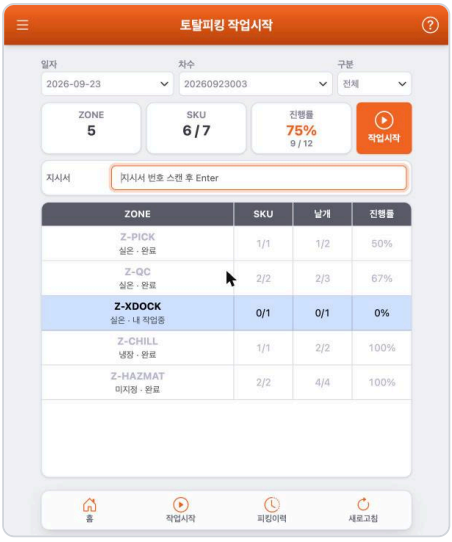
주문별 작업내역. 위 그리드에서 주문을 고르면 아래 그리드에 상품별 작업을 표시



DPC 카트 PAD. 상품을 스캔하면 담을 칸과 수량을 크게 보여 주고, 카트 칸 배치 그대로 진행 상태(담을 칸 · 완료 · 대기)를 색으로 표시



모바일 메인화면. 할 일 · 다음 작업 · 금일 예외를 먼저 보여 주고 메뉴는 타일로 배치



PDA 토탈피킹. 지시서 스캔으로 시작하고 ZONE 별 진행률을 표시



검수 PDA. 박스 → 상품 순서로 스캔하고 방금 처리한 내역을 아래에 표시

PC 화면

- Bootstrap 5.3 + jQuery 3.7, 그리드는 오픈소스 SlickGrid 5.19(MIT). 편집기 · 포매터 · 엑셀 · 팝업 · 열 설정 · 작업 버튼을 공용 모듈 6개로 분리
- 엑셀 입출력은 SheetJS, 표시기 화면의 실시간 수신은 브라우저 MQTT.js
- 탭으로 여러 화면을 열어 두는 구조, 즐겨찾기, 메뉴는 JSP 가 아니라 DB 테이블에서 그림
- 3D 현황은 three.js r186(MIT), 카드 차트는 라이브러리 없이 SVG 로 입체를 그리는 공용 모듈
- CDN 을 쓰지 않고 라이브러리 10종을 저장소에 두며 라이선스 파일을 함께 동봉

PDA · PAD · 키오스크

- PDA: 토탈피킹, 입고적치, 재고조사, 검수, DPS 보충, 반품, 수작업. 스캔 입력 한 칸과 큰 버튼 중심
- PAD: DAS 호기 PAD, DPC 카트 PAD(카트 칸 배치대로 적재 위치 표시), 수작업 PAD
- 키오스크: DPS 주문 스캔 점등 · 박스 마감
- 모바일 로그인 계정은 KPI · 다음 작업 · 예외를 보여 주는 전용 메인화면으로 진입

JSP 125개 가운데 모바일 화면 36개

AI 에이전트 기반 개발 방식 Claude Code 기반

무엇을 할지와 되돌릴 수 없는 결정은 사람이 하고, 에이전트는 정해진 절차로 실행합니다. 규칙은 문서와 함께 훅 · 게이트 스크립트 · 빌드 검사로 만들어, 지켜지지 않으면 다음 단계로 넘어가지 못합니다.

01

요구 확인 · 설계

질문으로 범위를 좁힌 뒤 설계, 번호 붙은 백로그

02

작업 브랜치 · 계획

main 직접 커밋 금지, 단계별 계획 문서

03

구현 · 검증

빌드 · 테스트 · 4개 언어 패리티 통과가 완료 조건

04

코드리뷰

남긴 발견 사항은 이슈로 등록

05

QA 5항목

격리 스택에 띄워 실측, 위험 항목만 사람 승인

06

PR · 자동 머지

CI 통과 뒤 GitHub 이 머지, 운영 배포만 사람

항목	WMS 본체 (2026-06 ~ 09)	레거시 이관 WMS (2026-09 ~)
작업 순서	13단계, 번호 → 계획 → 승인 → 개발(BE + FE + 다국어 한 PR) → 로컬 CI → 리뷰 → QA 5항목(3회 실패 시 중단) → ship	설계 → 브랜치 → 계획 → 구현 → 검증 → 리뷰 → QA 5항목 → PR → 자동 머지
강제 장치	훅 10개(답변 길이 상한, 커밋 전 리뷰, QA 게이트, 무한 대기 금지, i18n 패리티), 규칙 11, 스킬 5, 명령 3. 규칙 점검 스크립트(18개 검사)를 CI 에서 실행	명령 차단 훅 6종, 단계 게이트 4개(commit · push · PR · 종료), 빌드 가드(console.log · CDN · 이슈 번호 없는 TODO 금지, 소스 바꾸면 테스트도)
도구	자체 MCP 서버 6개(백로그, DB 읽기 전용, 테스트 러너, 시스템 상태, worktree, 프론트 자동화, TypeScript 4,223줄), QA 러너 에이전트 (Playwright), 백로그 자동 진행 명령	공식 플러그인(계획 · 검증 · 리뷰 스킬, 명령 가드), 이슈 템플릿 · 라벨, 날짜 브랜치 일괄 해결
병렬 작업	worktree 로 세션 여러 개를 격리, DB 백로그와 GitHub 연결	단일 세션, 두 머신(macOS · Windows) 공통 규칙
결과	PR 1,819 · QA 리포트 126건 · 교정 61건 중 41건을 분석해 규칙을 압축	PR 220 · 이슈 241(리뷰 발견 194건 이슈화) · 최종검증 날짜를 단 문서 33개

사람이 정하고 승인한 것

- 작업 범위와 순서, 질문해야 하는 결정의 기준(10점 척도)
- 되돌릴 수 없는 조작 · 외부 발송 · 운영 영향은 반드시 승인
- 규칙 파일 · 게이트 스크립트 · CI 설정 변경은 승인 후에만
- 운영 배포와 운영 DB 반영은 사람이 직접 실행
- 백로그 우선순위, 이슈 해결 순서

에이전트가 절차에 따라 실행한 것

- 설계 질문 → 계획 → 구현 → 테스트 작성 → 다국어 리소스
- 빌드 검증, 코드리뷰 요청, 발견 사항 이슈 등록
- QA 스택 기동 · 5항목 실측 · 리포트를 PR 본문에 기재
- PR 생성과 자동 머지 설정, 머지 뒤 로컬 반영
- 이슈를 날짜 브랜치로 묶어 커밋마다 이슈 번호 연결

에이전트 설정 자산 저장소 7개 기준

지침 · 규칙 · 스킬 · 훅 · 스크립트를 저장소에 파일로 두고 코드와 같이 리뷰하고 버전을 관리합니다.

마크다운 문서
427
파일 · 75,683줄

셸 스크립트
89
파일 · 12,145줄

훅 · 게이트
39
저장소 5개

규칙 · 스킬 · 명령
21 · 10 · 7
에이전트 지침 파일

종류	수량	내용
지침 문서	프로젝트 4 + 전역 1	명령 · 구조 · 작업 범위 · 작업 순서 · 검증 · 커밋 규칙. 전역 지침은 저장소에 정본을 두고 macOS · Windows 두 머신이 같은 내용을 씀
규칙 파일	21	백엔드 Java, MyBatis 매퍼 · 벤더별 SQL, 프론트 화면, QA 게이트, 검증, 백로그, DB 반영, 이슈 흐름, 외부 라이브러리 라이선스, 문서 관리. 해당 경로의 파일을 읽을 때만 불러오도록 경로 조건을 둠
스킬	10	작업 시작 · 마무리 · PR 전 리뷰 · QA 게이트 실행 · 머지 후 반영, 새 화면 생성(JSP · 컨트롤러 · 매퍼 · 메뉴 한 번에), 표시기 · Redis 이식 패키지, Unity 테스트 실행, 특허 조사
명령 · 에이전트	7 · 1	백로그 시작 · 백로그 자동 진행 · ship(검증 → PR → 머지), 교정 등록 · 교정 통계 · 규칙 점검, QA 러너 에이전트
훅 · 게이트	39	세션 시작 때 상태 주입, 커밋 전 리뷰, QA 게이트, 무한 대기 금지, 답변 길이 상한, 다국어 패리티, 되돌릴 수 없는 명령 · 프로세스 종료 · 종료코드를 가리는 파이프 · 비밀 설정 편집 차단, 승인된 계획서 없는 커밋 차단, 생성 폴더 편집 차단
셸 스크립트	89	QA 격리 스택 기동 · 종료 · 항목 해시, 로컬 CI 범위 계산, 스키마 덤프 · 운영 스키마 대조, 배포 · 원격 재시작, 브랜치 보호 설정, 러너 감시(watchdog), worktree 점검, ship · merge
자체 제작 플러그인	2	교정 플러그인(훅 13 · 명령 3, 마켓플레이스 형식으로 배포), 특허 조사 스킬 플러그인(특허 API 조회 스크립트 포함)
자체 MCP 서버	6	백로그, DB 읽기 전용, 테스트 러너, 시스템 상태, worktree, 프론트 자동화
도입한 공식 플러그인	12	계획 · 검증 · 리뷰 스킬 묶음, 훅 규칙 작성, 코드리뷰, 보안 검사, 지침 문서 점검, 라이브러리 문서 조회, 브라우저 자동화, C# · Java 언어 서버, Unity 연동
백그라운드 자동화	1	PR 이 머지되면 5분 주기 작업이 로컬 main 을 맞추고 연결된 이슈를 닫음

마크다운 문서 427개의 구성

- WMS 본체 283개(35,191줄): 설계 결정 기록(ADR) 11, 상세설계 14, QA 리포트, 규칙 · 스킬 · 명령, 운영 절차
- 레거시 이관 WMS 106개(23,976줄): 업무별 설계 문서(위치재고 · 총량피킹 · 입고적치 · 재고조사 · DPS · DPC · 요율 · 박스 적입 수량 · 3D 현황), 기능별 설계 10건 · 구현 계획 16건, 특허 조사 2건, 표시기 MQTT 토픽, Redis 캐시 · 세션, 라벨 인쇄, CI 가드
- 디지털 트윈 10개(11,982줄): 설계 1건과 구현 계획 6건
- 연동 미들웨어 18개(4,214줄), 플러그인 · 특허 검색 10개

문서를 쓰는 규칙

- 지시는 지침 · 규칙 파일에, 절차 · 설계 · 조사 결과는 문서 폴더에 둠. 같은 내용을 두 곳에 적지 않음
- 미결 사항은 문서에 적지 않고 이슈로 등록, 문서에는 확정된 사실과 근거만 남김
- 실측한 사실에는 날짜를 붙임(예: 어떤 설정이 실제로 적용되는지 확인한 날)
- 규칙을 정하면 검사로 고정하고, 한 번 깨뜨려 실패가 나는지 확인한 뒤 반영

교정 플러그인: 에이전트 실수 기록과 차단

- 에이전트가 확인하지 않고 말한 진술, 규칙 위반을 유형별(단정 · 원인 역전 · 파괴적 명령 · 약속 · 문서 · 절차) 이슈로 쌓고, 세션 시작 때 요약을 주입해 세션당 새 이슈 수로 반복 여부를 잽니다. 2026-08-29 13건에서 시작해 09-26 까지 195건 축적
- 분석 결과: 확인 없이 단정한 답변 41건 중 문장 패턴으로 잡힌 것은 8건(20%)이었고, 놓친 33건 중 24건은 명령을 하나도 실행하지 않고 답한 경우였습니다. 그래서 턴마다 조회 도구 사용 여부를 추적하는 방식으로 바꿨습니다

AI 공동작성 커밋

2,403 / 4,018

저장소 9개, 머지 제외

QA 리포트

126 + 95

본체 파일 · 이관 PR 본문 QA 절

교정 이슈

195

2026-08-29 ~ 09-26

에이전트가 쓴 코드도 사람이 쓴 코드와 같은 게이트 · 리뷰 · CI 를 통과해야 main 에 들어갑니다.

CI/CD 자동화

PR 부터 운영 서버 배포까지, 서비스 5개 공통

01

로컬 게이트

commit · push 시 빌드 가드 · 테스트 · 규칙 점검

02

GitHub Actions

self-hosted 러너. 백엔드 · 프론트 · 통합(Testcontainers) 잡, 실패 시 리포트 업로드

03

main 보호 · 자동 머지

CI 통과 필수, 관리자도 예외 없음. PR 생성 직후 자동 머지 설정

04

Jenkins

main 변경 감지 → 테스트 → 번들 jar/war 빌드 → 라이선스 고지 · SPA 포함 검증 → 보관

05

스키마 게이트

운영 미반영 SQL 이 남아 있으면 배포 중단

06

배포 · 헬스체크

산출물 교체 → systemd 재시작 → 상태 URL 200 확인, 실패 시 빌드 실패

- 본체 · EAI · 분석 · 이관 WMS · 가계부 · 특허 검색 모두 Jenkins → systemd 로 같은 방식으로 배포합니다. 테스트 실패 시 배포하지 않습니다.
- 통합 테스트는 격리된 MariaDB · Redis(Docker) 위에서 돌고, 하나라도 건너뛰면 빌드가 실패합니다.
- 로그인 세션 · 토큰을 Redis 에 둘 수 있게 해 재배포 후에도 사용자가 다시 로그인하지 않도록 했습니다.
- 이관 WMS 는 테스트 순서를 무작위로 바꿔 한 번 더 돌리고, 두 프로젝트 모두 컴파일 경고를 오류로 처리합니다.

일하는 방식

물류 도메인 전반

입고 · 재고 · 출고 · 반품 · 구매 · 배차 · 설비(DAS · DPS · DPC · RFID) · SAP 연동까지 WMS 전 영역을 패키지 단위로 직접 설계하고 구현했습니다.

여러 언어 · 플랫폼

주 언어인 C# 에 더해 Java · Python · TypeScript 로 각각 운영되는 서비스를 만들었고, C# · MariaDB · iBATIS · Vue · 상용 리포트에서 Java · Oracle 구조로 옮기면서 원본 대비 변경점과 결함 보정 내역을 문서로 남겼습니다.

운영을 고려한 설계

커밋 후 외부 전송, 멱등 처리기와 아웃박스, 발행 큐 재시도, 배포 후 헬스체크, 스키마 게이트, 재배포 세션 유지를 설계 단계에서 넣습니다.

개발 체계 설계

작업 절차 · 리뷰 → 이슈 추적 · 품질 게이트 · CI/CD · AI 협업 규칙을 직접 설계해 넉 달간 운영했고, 팀에 그대로 적용할 수 있습니다.

팀에 적용할 것

- 팀 규칙을 문서와 자동 검사로 함께 운영해, 리뷰에서 같은 지적이 반복되지 않게 함
- AI 도구는 맡길 범위와 사람이 승인할 범위를 먼저 정하고, 결과물은 같은 품질 기준으로 검증
- 레거시 전환은 원본 대비 변경점을 기록하며 단계적으로 진행
- 리뷰 · QA 에서 나온 미결 사항은 이슈로 관리

수치는 레거시 이관 WMS 2026-10-05, 그 외 저장소 2026-10-01 기준입니다. 소스 코드는 요청하시면 화면 공유로 보여드립니다.